

Retrieval is Cheap, Show Me the Code: Executable Multi-Hop Reasoning for Retrieval-Augmented Generation

Jiashuo Sun^{*}  Jimeng Shi^{*}  Yixuan Xie  Saizhuo Wang  Jash Rajesh Parekh 
 Pengcheng Jiang  Zhiyi Shi  Jiajun Fan  Qinglong Zheng  Peiran Li 
 Shaowen Wang  Ge Liu  Jiawei Han 

 University of Illinois Urbana-Champaign

 Hong Kong University of Science and Technology

 Texas A&M University

 GitHub

 Project Page

 Model

Abstract

Retrieval-Augmented Generation (RAG) has become a standard approach for knowledge-intensive question answering, but existing systems remain brittle on multi-hop questions, where solving the task requires chaining multiple retrieval and reasoning steps. Key challenges are that current methods represent reasoning through free-form natural language, where intermediate states are implicit, retrieval queries can drift from intended entities, and errors are detected by the same model that produces them making self-reflection an unreliable, ungrounded signal. We observe that multi-hop question answering is a typical form of step-by-step computation, and that this structured process aligns closely with how code-specialized language models are trained to operate. Motivated by this, we introduce **PyRAG**, a framework that reformulates multi-hop RAG as program synthesis and execution. Instead of free-form reasoning trajectories, **PyRAG** represents the reasoning process as an executable Python program over retrieval and QA tools, exposing intermediate states as variables, producing deterministic feedback through execution, and yielding an inspectable trace of the entire reasoning process. This formulation further enables compiler-grounded self-repair and execution-driven adaptive retrieval without any additional training. Experiments on five QA benchmarks (PopQA, HotpotQA, 2WikiMultihopQA, MuSiQue, and Bamboogle) show that **PyRAG** consistently outperforms strong baselines under both training-free and RL-trained settings, with especially large gains on compositional multi-hop datasets. Our code, data and models are publicly available at <https://github.com/GasolSun36/PyRAG>.

1 Introduction

Retrieval-Augmented Generation (RAG) [8, 23] has emerged as a foundational paradigm for knowledge-intensive question answering, allowing large language models (LLMs) to ground their outputs in external evidence and produce more factual responses [6, 11]. While vanilla RAG works well for single-hop queries, many real-world questions require multi-hop reasoning [12, 48, 43, 30, 35], where the answer must be assembled by chaining evidence across multiple sources. For example, answering “Who is older, Jed Hoyer or John William Henry II?” requires retrieving two birth dates,

^{*}Equal contribution.

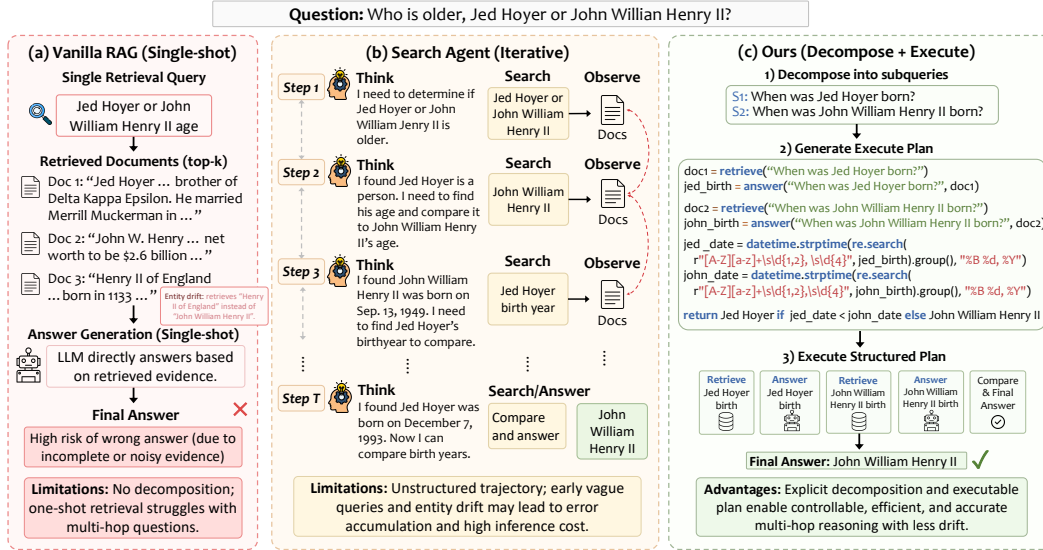


Figure 1: **Comparison across Vanilla RAG, Search Agents, and PyRAG (Ours).** Given the multi-hop question “Who is older, Jed Hoyer or John William Henry II?”, (a) Vanilla RAG performs single-shot retrieval and is prone to incomplete or noisy evidence; (b) Search Agents follow an unstructured iterative trajectory where vague queries and entity drift (e.g., retrieving “Henry II of England” instead of “John William Henry II”) accumulate errors across steps; (c) **PyRAG** decomposes the question into atomic sub-queries and generates an executable program that retrieves, answers, and composes intermediate results through explicit variables, yielding a controllable, inspectable, and accurate reasoning process.

maintaining them as intermediate results, and composing them through an explicit comparison. Such questions are pervasive in open-domain QA and stress-test a system’s ability to plan, retrieve iteratively, and aggregate evidence across steps. Figure 1 illustrates how three representative paradigms, Vanilla RAG, Search Agents, and our **PyRAG**, approach this question, highlighting the structural differences in how each maintains intermediate state and composes evidence.

Existing multi-hop RAG approaches are typically achieved via free-form natural language reasoning, including chain-of-thought prompting [45], iterative retrieve-and-reason loops [44, 49, 31, 35], and more recently, reinforcement-learned search agents [17, 36, 50, 2]. While these methods introduce decomposition and iteration, the reasoning state remains implicit in text: intermediate results are embedded in narrative form rather than maintained as discrete objects, retrieval queries can drift from the intended entities (e.g., querying “Henry II of England” when the question concerns “John William Henry II”), and errors are detected by the same LLM that produces them, turning self-reflection into an unreliable, ungrounded signal. As a result, the reasoning trajectory is hard to control, verify, and troubleshoot. Although a parallel line of program-guided reasoning work [7, 3, 4, 25, 27, 28] does leverage executable code, they assume that the evidence required for reasoning is available a priori in self-contained inputs such as tables or closed corpora. This assumption breaks down in open-domain multi-hop QA, where intermediate answers are unknown at synthesis time, and subsequent queries must depend on the results of earlier retrievals. Table 1 summarizes how these reasoning paradigms differ along five key dimensions: multi-hop capability, interpretability, structured planning, reflection, and executable interface, motivating the design of **PyRAG** as a paradigm that supports all five.

We argue that the root cause of these limitations is a mismatch between task structure and reasoning representation. Multi-hop question answering is fundamentally a form of step-by-step computation: it decomposes a question into sub-problems, computes intermediate results, and composes them through explicit dependencies. This process mirrors how programs are constructed and executed, a sequence of operations over named variables, connected by data flow. Yet, current methods simply encode this structured computation into unstructured natural language by forcing the LLM to simultaneously plan, maintain state, and reason. We further observe that code-specialized language models are explicitly trained for this exact pattern of behavior: maintaining intermediate variables, enforcing control flow,

Table 1: **Comparison of reasoning paradigms.** Search Agents are marked partial on Structured Planning because their plans are implicit and unfold reactively in the thought trace rather than being materialized as an explicit, inspectable artifact, and partial on Reflection because error signals come from the LLM’s own self-judgment rather than grounded external feedback. They lack an Executable Interface entirely, as the reasoning trajectory is natural language with no variables, data flow, or deterministic execution. Our **PyRAG** addresses all three limitations by representing the multi-hop reasoning process as an executable program.

Paradigm	Multi-hop Capability	Interpretability	Structured Planning	Reflection	Executable Interface
Vanilla RAG (Single-shot)	✗	✗	✗	✗	✗
Search Agent (Free-form)	✓	✓	△	△	✗
PyRAG (Executable program)	✓	✓	✓	✓	✓

✓: supported △: partially supported ✗: not supported

and producing step-by-step structured programs [14]. This suggests a natural reformulation: if we represent multi-hop reasoning as program synthesis rather than free-form generation, we can directly leverage the inductive bias of code models, while simultaneously gaining explicit state, deterministic feedback from execution, and an inspectable trace of the reasoning process.

Motivated by this observation, we introduce **PyRAG**, a framework that provides a verifiable execution interface for multi-hop RAG. **PyRAG** casts multi-hop reasoning as the synthesis and execution of a Python program over a small set of tool APIs: `retrieve(query)` and `answer(query, docs)`, where each step retrieves evidence, computes an intermediate answer, and stores the result as a variable that can be reused downstream. The framework consists of three specialized agents: a Decompose Agent that breaks the input question into atomic sub-queries, a Plan Agent that translates the sub-queries into an executable program, and an Answer Agent that produces short answers from retrieved evidence. Crucially, the executable formulation gives rise to two natural refinement mechanisms with no additional training: a *compiler-grounded self-repair loop*, where runtime exceptions provide deterministic signals for the Plan Agent to revise the program, and an *execution-driven adaptive retrieval* mechanism that selectively increases the retrieval scope when an intermediate answer indicates insufficient evidence. Both arise directly from the program-execution interface rather than relying on LLM self-reflection.

We evaluate **PyRAG** on five open-domain QA benchmarks (PopQA, HotpotQA, 2WikiMultihopQA, MuSiQue, Bamboogle) under both training-free and RL-trained settings. Our contributions are:

- We identify a structural mismatch between multi-hop reasoning and its representation in existing RAG systems, and reformulate multi-hop QA as an **executable** step-by-step process.
- We introduce **PyRAG**, a framework that provides a verifiable execution interface with explicit state, deterministic compiler feedback, and inspectable reasoning traces, equipped with execution-guided self-repair and adaptive retrieval.
- We show that the advantage of code-specialized models is task-dependent: it emerges only under program-synthesis interfaces, highlighting that model capability and reasoning interface must be co-designed.
- Empirically, **PyRAG** improves over Vanilla RAG by +11.8 average EM (training-free, 7B) and +25.5 on Bamboogle, while **PyRAG**-RL achieves the highest average EM among 7B-scale RL-trained methods and generalizes across Qwen3-4B and LLaMA-3.1-8B backbones.

2 Method

2.1 Overview

We present **PyRAG**, a framework that introduces an executable interface for multi-hop RAG, as shown in Figure 2. Instead of representing reasoning as free-form natural language, **PyRAG** decomposes the problem into a sequence of structured steps and executes them through a program.

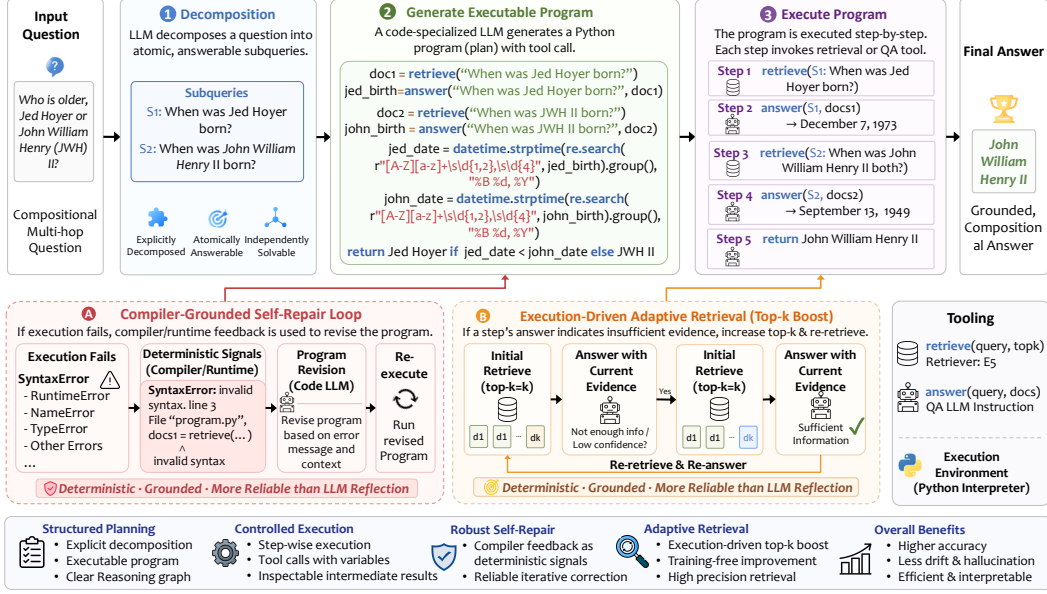


Figure 2: **The PyRAG framework.** Given a multi-hop question, **PyRAG** proceeds in three stages: (1) **Decompose**: an LLM breaks the question into atomic, independently answerable sub-queries; (2) **Plan**: a code-specialized LLM synthesizes an executable Python program over two tool primitives, `retrieve(query, topk)` and `answer(query, docs)`, where intermediate results are bound to variables and composed through explicit data dependencies; (3) **Execute**: the program is run step-by-step in a Python interpreter, producing an inspectable trace and a grounded final answer. Two execution-guided refinement mechanisms refine this pipeline: (A) **Compiler-Grounded Self-Repair**, which uses runtime exceptions (e.g., `SyntaxError`, `NameError`) as deterministic signals for the planner to revise and re-execute the program; and (B) **Execution-Driven Adaptive Retrieval**, which boosts the top- k retrieval budget for sub-steps whose answer indicates insufficient evidence. Both mechanisms are training-free and rely on grounded execution feedback rather than LLM self-reflection.

Given a question q , **PyRAG** consists of three components: (1) a **decomposition agent** that breaks q into atomic sub-queries, (2) a **planning agent** that generates an executable program describing the reasoning process, and (3) an **answer agent** that produces answers based on retrieved evidence.

At the inference time, the generated program is executed step-by-step, where each step corresponds to a retrieval or question-answering operation. Therefore, we shift multi-hop reasoning from opaque narrative to an explicit, controllable, and verifiable execution process.

2.2 Motivation: Multi-Hop QA as Step-by-Step Computation

We argue that multi-hop question answering can be naturally viewed as a form of step-by-step computation. Resolving multi-hop queries necessitates a systematic decomposition into constituent sub-problems, the computation of intermediate results, and the ultimate synthesis of these findings into a final answer [44, 49, 31, 45].

This structured process closely aligns with the fundamental principles of programmatic execution: A program defines a sequence of functional operations, maintains intermediate variables, and enforces dependencies between steps [14]. Code-specialized language models are explicitly trained for such behavior. They are optimized to generate structured programs that decompose tasks, maintain state through variables, and perform consistent step-by-step execution. As a result, they provide a strong inductive bias for multi-hop QA processing.

Motivated by this observation, we cast multi-hop RAG as a program synthesis problem, where the reasoning process is represented as an executable plan. This allows us to directly leverage the step-by-step reasoning capability of code models for explicit control and verification, rather than forcing it to emerge from free-form natural language reasoning.

2.3 PyRAG Agents

Decomposition Agent. Given a question q , the decomposition agent produces a sequence of sub-queries $s = [s_1, \dots, s_n]$, where each sub-query is designed to be answerable with a single retrieval step. This step introduces an explicit structure over the reasoning process, but does not yet define how the steps should be executed or combined.

Answer Agent. The answer agent takes a sub-query and a set of retrieved documents as input, and produces a short answer. It is implemented using an instruction-following LLM, and is responsible for extracting information from retrieved evidence and performing final aggregation.

Planning Agent. The planning agent is the core component of PyRAG. Given the original question q and the decomposed sub-queries s , it generates a program π that specifies how to solve the task through a sequence of retrieval and answering operations.

2.4 Executable Planning

We define two APIs for the execution tool:

- `retrieve(query, topk=k)`: returns the top-k relevant documents for the given query, where k can be increased adaptively at execution time (Sec. 2.6).
- `answer(query, docs)`: returns an answer conditioned on documents.

The planning agent generates a program that composes these APIs through variable assignments. Each step retrieves evidence, computes an intermediate answer, and stores the result in a variable. These variables are then reused in subsequent steps.

This formulation makes the reasoning process explicit: instead of implicitly encoding intermediate states in text, the program stores them as variables and connects them through data dependencies. The final answer is produced by aggregating these intermediate results.

2.5 Execution

The generated program π is executed step-by-step. At each step, the system invokes either `retrieve()` or `answer()`, and stores the output for later use.

This execution process yields an execution trace, which records all intermediate queries, retrieved documents, and answers. The trace provides a transparent view of the reasoning process and enables debugging and analysis.

2.6 Execution-Guided Reflexion

A key advantage of executable planning is that it naturally supports refinement during execution.

Compiler-Grounded Self-Repair If the generated program fails to execute due to invalid operations or inconsistent variable usage, the execution environment returns a structured error signal. The planning agent can then revise the program based on this feedback and re-execute it.

Adaptive Retrieval If an intermediate answer indicates insufficient evidence, the system can selectively increase the retrieval scope for that step and re-run the corresponding operation. This allows targeted correction without modifying the entire reasoning plan.

These mechanisms arise naturally from the executable formulation, without requiring additional training or specialized control logic.

Table 2: Exact Match (%) of **training-free** methods on five QA benchmarks. All methods use the same setting and identical evaluation splits. Best result within each backbone size is in **bold**; second best is underlined. Δ shows improvement over Vanilla RAG within the same backbone.

Method	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	Avg.
<i>Backbone: Qwen2.5-7B-Instruct</i>						
Direct Inference	14.0	18.3	12.6	3.1	12.0	12.0
CoT [45]	5.4	9.2	10.8	2.2	23.2	10.2
Vanilla RAG	26.7	28.9	18.9	4.7	16.0	19.0
Self-Ask [30]	29.4	30.2	21.5	6.8	22.1	22.0
IRCoT [44]	<u>32.6</u>	<u>32.7</u>	24.8	<u>9.1</u>	24.3	24.7
ITER-RETGEN [31]	31.4	32.5	<u>28.9</u>	8.7	<u>29.6</u>	<u>26.2</u>
PyRAG (ours)	33.5	34.0	33.4	11.8	41.5	30.8
Δ vs. Vanilla RAG	+6.8	+5.1	+14.5	+7.1	+25.5	+11.8
<i>Backbone: Qwen2.5-72B-Instruct</i>						
Direct Inference	19.7	30.6	20.6	5.5	17.6	18.8
CoT [45]	24.4	33.2	25.4	9.9	19.6	22.5
Vanilla RAG	33.2	36.8	30.4	10.6	21.6	26.5
Self-Ask [30]	41.4	48.2	32.5	11.8	26.1	32.0
IRCoT [44]	<u>44.6</u>	<u>50.9</u>	35.8	<u>14.2</u>	28.3	34.8
ITER-RETGEN [31]	43.4	50.5	<u>40.2</u>	13.8	<u>33.6</u>	<u>36.3</u>
PyRAG (ours)	45.5	52.0	44.4	16.9	45.5	40.9
Δ vs. Vanilla RAG	+12.3	+15.2	+14.0	+6.3	+23.9	+14.4

3 Experiments

3.1 Experimental Setup

Benchmarks. We evaluate on five open-domain QA benchmarks spanning single-hop and multi-hop reasoning: PopQA [26], HotpotQA [48], 2WikiMultihopQA [12], MuSiQue [43], and Bamboogle [30].

Exact Match (EM) is used as the primary metric for all benchmarks. HotpotQA serves as the in-domain training set for RL-trained variants; all remaining datasets are evaluated out-of-domain.

Baselines. We compare against the following categories of methods:

Training-free baselines. **Direct Inference** and **CoT** [45] require no retrieval. **Vanilla RAG** [23] performs single-step retrieve-then-read. **Self-Ask** [30] decomposes questions into sub-questions with interleaved retrieval. **IRCoT** [44] interleaves chain-of-thought reasoning with iterative retrieval. **ITER-RETGEN** [31] alternates between retrieval and generation across multiple rounds.

RL-trained baselines. **RAG-SFT** and **RAG-RL** are supervised fine-tuning and reinforcement learning variants of a standard RAG pipeline. **ZEROSEARCH** [37], **Search-R1** [17], **StepSearch** [50], and **ReSearch** [2] are recent RL-based methods that train models to perform adaptive retrieval.

Our methods. **PyRAG** is our training-free multi-agent framework; **PyRAG-RL** further fine-tunes the framework with reinforcement learning. Unless stated otherwise, all **PyRAG** variants use Qwen2.5-7B-Instruct as the backbone.

Implementation Details. We follow the retrieval and data setup of Search-R1 [17] exactly: an E5-base dense retriever over the Wikipedia 2018 dump [18], with the same training splits and evaluation data preprocessing. The default number of retrieved passages per sub-query is $k=5$. When an `answer()` call returns an insufficient-information response (e.g. “unknown” or “cannot answer”), the runner automatically re-executes the same code with an increased retrieval budget of $k=10$ for the implicated steps. Additional implementation details including training are provided in Appendix E.1.

Table 3: Exact Match (%) of RL-trained methods on four QA benchmarks. All methods are evaluated under the same retrieval setting. The best result within each backbone is in **bold**. [†] denotes in-domain evaluation; remaining datasets are out-of-domain.

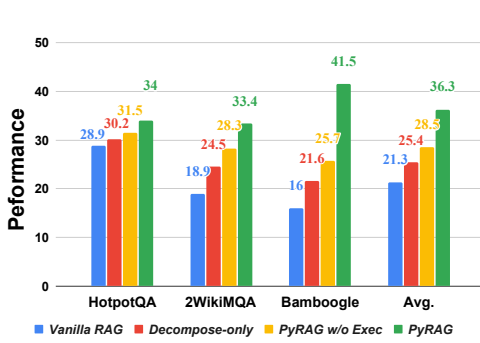
Method	HotpotQA [†]	2WikiMQA	MuSiQue	Bamboogle	Avg.
<i>Backbone: Qwen2.5-7B-Instruct</i>					
Vanilla RAG	28.9	18.9	4.7	16.0	21.3
RAG-SFT	32.4	22.6	6.8	27.1	22.2
RAG-RL	35.2	34.7	9.6	29.6	27.3
ZEROSEARCH [37]	34.6	35.2	18.4	27.7	29.0
Search-R1 [17]	37.0	41.4	14.6	36.8	32.4
StepSearch [50]	38.6	36.6	22.6	40.0	34.5
ReSearch [2]	43.5	47.6	22.3	42.4	38.9
PyRAG-RL (ours)	40.5	49.4	20.7	46.1	39.2
<i>Backbone: Qwen3-4B-Instruct</i>					
Vanilla RAG	27.1	16.7	4.3	14.8	15.7
RAG-SFT	30.5	20.1	6.2	25.4	20.6
RAG-RL	33.2	31.8	8.8	27.6	25.4
PyRAG-RL (ours)	38.4	45.1	18.6	43.2	36.3
<i>Backbone: LLaMA-3.1-8B-Instruct</i>					
Vanilla RAG	30.3	19.4	6.3	17.6	18.4
RAG-SFT	34.1	23.3	8.5	29.3	23.8
RAG-RL	37.5	35.2	11.4	31.8	29.0
PyRAG-RL (ours)	43.2	50.1	22.1	48.3	40.9

3.2 Main Results

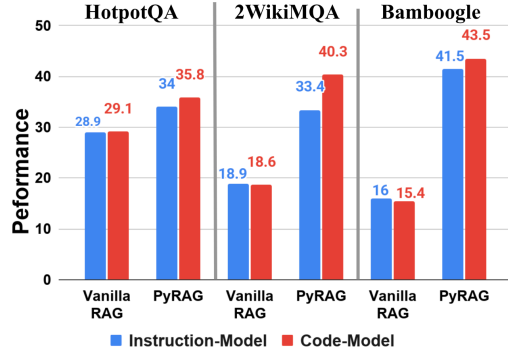
Table 2 and Table 3 report the main results under training-free and RL-trained settings, respectively.

Training-free results. Under the training-free setting (Table 2), **PyRAG** consistently outperforms all baselines across both backbone sizes. With Qwen2.5-7B-Instruct, **PyRAG** achieves an average EM of 30.8, surpassing the strongest baseline ITER-RETGEN by +4.6 points and Vanilla RAG by +11.8 points. Gains are most pronounced on compositional multi-hop benchmarks: +14.5 on 2WikiMQA and +25.5 on Bamboogle relative to Vanilla RAG, datasets specifically designed to stress systems that cannot chain multiple retrieval steps. On PopQA and HotpotQA, **PyRAG** also achieves the best results (33.5 and 34.0), demonstrating that the structured decompose-plan-answer pipeline does not degrade performance on relatively simpler queries. Scaling to Qwen2.5-72B-Instruct amplifies these trends: **PyRAG** reaches an average of 40.9, outperforming ITER-RETGEN by +4.6 and delivering the largest single-dataset gain on Bamboogle (+23.9 over Vanilla RAG).

RL-trained results. Table 3 compares **PyRAG** trained with reinforcement learning (**PyRAG-RL**) against competitive RL- and SFT-based baselines. With the Qwen2.5-7B backbone, **PyRAG-RL** achieves an average EM of 39.2, on par with ReSearch (38.9) while outperforming all other baselines including Search-R1 (+6.8) and StepSearch (+4.7). Notably, **PyRAG-RL** attains the highest score on 2WikiMQA (49.4) and Bamboogle (46.1) among 7B models, while remaining competitive on the HotpotQA and MuSiQue. **PyRAG-RL** generalizes well across architectures: it achieves 36.3 average EM on Qwen3-4B and 40.9 on LLaMA-3.1-8B, consistently surpassing the corresponding RAG-RL baselines by +10.9 and +11.9 points, respectively, confirming that the structured planning prior of **PyRAG** translates effectively to the RL fine-tuning regime.



(a) Adding decomposition, planning, and execution to Vanilla RAG yields monotonic gains from 21.3 to 36.3 average EM, with execution contributing the largest jump.



(b) Code-specialized models show negligible advantage under Vanilla RAG but consistent gains under PyRAG, indicating that their benefit is task-dependent and emerges only when reasoning is formulated as program synthesis.

3.3 Ablation Study

Progressive Component To understand the contribution of each component in PyRAG, we perform an ablation study that progressively introduces structure into the reasoning process. As shown in Figure 3a, we observe a consistent improvement from Vanilla RAG to PyRAG across all three multi-hop benchmarks.

Introducing explicit decomposition (Decompose-only) yields modest gains over Vanilla RAG, indicating that breaking down complex questions into sub-queries already improves retrieval quality. However, representing the reasoning process as a structured plan (PyRAG w/o execution) leads to further improvements, suggesting that organizing intermediate steps—even without execution—helps guide the model toward more coherent reasoning.

The largest gains are achieved by PyRAG, which compiles and executes the generated plan as an executable program. This result highlights the importance of execution-based reasoning, where intermediate results are explicitly computed and passed across steps, rather than implicitly inferred.

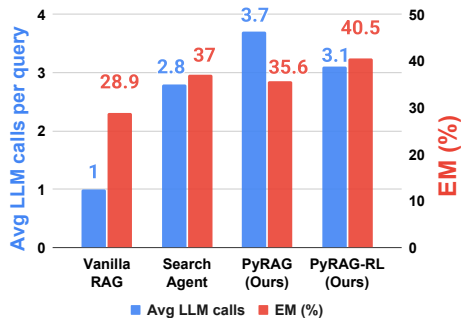
Effect of Model Specialization. We further investigate whether PyRAG’s gains arise from improved model capability or from the proposed planning framework. As shown in Figure 3b, under Vanilla RAG, replacing the instruction-tuned model with a code-specialized counterpart yields negligible differences across all three benchmarks (e.g., 28.9 vs. 29.1 on HotpotQA, 18.9 vs. 18.6 on 2WikiMQA), indicating that code-specialized models offer no general advantage in standard RAG. Under PyRAG, however, the code-specialized model consistently outperforms the instruction-tuned counterpart, with the gap widening on harder multi-hop benchmarks (+1.8 on HotpotQA, +6.9 on 2WikiMQA, +2.0 on Bamboogle). Notably, even the instruction-model variant of PyRAG already substantially outperforms Vanilla RAG, confirming that the gains come primarily from structured planning, with code specialization providing additional task-aligned leverage. This indicates that model capability and reasoning interface must be co-designed: code models’ strengths are realized only when reasoning is explicitly formulated as program synthesis.

3.4 Analysis

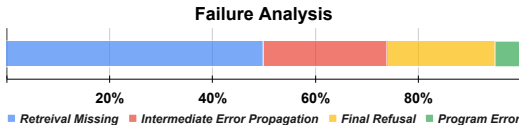
Efficiency Analysis We compare PyRAG against representative baselines in both EM and inference cost, measured as the average number of LLM calls per query over 100 randomly sampled HotpotQA queries; we select Search-R1 as the strongest RL-trained search agent baseline.¹

As shown in Figure 4a, Vanilla RAG is cheapest (one call) but performs poorly on multi-hop questions, while Search-R1 improves accuracy through unstructured iterative retrieval. PyRAG matches Search-R1’s EM with a modest 3.7-call average, of which compiler-grounded self-repair triggers on ~5% of

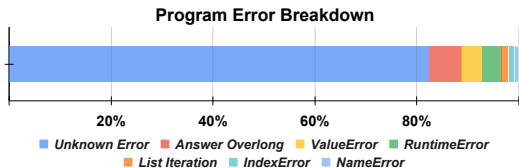
¹For PyRAG, the Decompose and Plan stages are merged into a single LLM call; reported counts comprise this planning call plus all `answer()` invocations and any self-repair or adaptive-retrieval re-executions.



(a) **PyRAG** achieves comparable EM to Search Agent with a modest increase in LLM calls, while **PyRAG-RL** attains the highest EM with fewer calls than the training-free **PyRAG**, indicating that RL fine-tuning produces more disciplined reasoning.



(b) The answer agent accounts for $\sim 95\%$ of failures, while program errors contribute only $\sim 5\%$, identifying the answer agent as the primary bottleneck.



(c) Among program errors, the dominant mode is Unknown Error, the program executes successfully but the answer agent fails to extract a grounded answer from retrieved evidence, rather than explicit runtime exceptions.

queries and execution-driven adaptive retrieval on $\sim 20\%$, indicating that under-evidenced sub-steps rather than malformed programs are the primary driver of re-executions. **PyRAG-RL** achieves the highest EM with even fewer calls (3.1 vs. 3.7): RL fine-tuning produces more targeted queries and triggers both refinement mechanisms less frequently as the policy becomes more reliable. Together, these results indicate that the program-based structure assigns each LLM call a well-defined role, yielding a better accuracy–cost trade-off than unstructured iterative baselines.

Failure Analysis To understand the error sources of **PyRAG**, we manually categorize 100 randomly sampled incorrect predictions from HotpotQA. As shown in Figure 4b, retrieval missing accounts for roughly half of all failures, identifying upstream retrieval recall as the dominant bottleneck. The next largest category is intermediate error propagation, where an uncertain sub-answer corrupts downstream steps (Failure F2), followed by final refusals where the answer agent declines despite the program executing as intended. Program errors contribute only $\sim 5\%$, confirming that the planning agent reliably produces well-formed executable code. We further characterize program errors among the same sampled cases (Figure 4c). The dominant mode is Unknown Error, in which the program executes without raising an exception but the answer agent returns a sentinel response (e.g., “unknown”) because it fails to compose an answer from the retrieved evidence—a context-utilization issue rather than a Python-level fault. Genuine runtime exceptions (ValueError, TypeError, IndexError, NameError) together account for less than 20% of program errors and are typically traceable to mismatched assumptions about retrieved string formats (e.g., Failure F5).

Case Study Due to space limitations, we defer the detailed case studies and qualitative examples to the appendix G.

4 Related Work

Multi-Hop Retrieval-Augmented Generation. Multi-hop QA requires chaining evidence across passages, which vanilla RAG [23] cannot handle in a single step. Prior work tackles this through iterative retrieve-and-reason prompting [45, 49, 30, 44, 16, 31, 19], graph-based reasoning over retrieved content [5, 10, 1, 29], and RL-trained search policies [17, 36, 50, 24]. In all of these, the retrieval–reasoning interaction remains an implicit trajectory and error detection relies on LLM self-judgment. **PyRAG** instead represents the full pipeline as an executable program, making reasoning explicit and verifiable via compiler feedback.

Program-Guided Reasoning. Executable code has proven effective for reasoning over well-defined symbolic structures [7, 3, 4, 25, 27, 28], but these approaches assume the evidence is available *a priori*

in self-contained inputs. A complementary line, exemplified by DSPy [20], treats LM pipelines as compilable programs and optimizes their prompts. **PyRAG** targets a different setting—open-domain multi-hop QA where intermediate answers are unknown at synthesis time and later queries depend on earlier results, and contributes a concrete program-execution interface (see Appendix C for extended discussion).

5 Conclusion

We presented **PyRAG**, a framework that reformulates multi-hop RAG as program synthesis and execution. By encoding the retrieval–reasoning process as an executable Python program, **PyRAG** exposes intermediate states as variables, produces deterministic compiler feedback, and yields an inspectable reasoning trace, while enabling training-free self-repair and adaptive retrieval as direct byproducts of the execution interface. Across five QA benchmarks under both training-free and RL-trained settings, **PyRAG** delivers consistent gains over strong baselines, with the largest improvements on compositional multi-hop datasets.

References

- [1] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. Pathrag: Pruning graph-based retrieval augmented generation with relational paths. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [2] Mingyang Chen, Linzhuang Sun, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- [3] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- [4] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, et al. Binding language models in symbolic languages. *arXiv preprint arXiv:2210.02875*, 2022.
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, 2024.
- [7] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International conference on machine learning*, 2023.
- [8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, Haofen Wang, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [10] Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 2024.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

- [12] Xanh Ho, Anh-Khoa Nguyen, Ehsan Abbasnejad, and Dinh Phung. Constructing a multi-hop question answering dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, 2020.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [14] Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, et al. Da-code: Agent data science code generation benchmark for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13487–13521, 2024.
- [15] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- [16] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023.
- [17] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [18] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, 2020.
- [19] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp. *arXiv preprint arXiv:2212.14024*, 2022.
- [20] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. 2024.
- [21] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 2019.
- [22] Woosuk Kwon. *vLLM: An Efficient Inference Engine for Large Language Models*. PhD thesis, UC Berkeley, 2025.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33, 2020.
- [24] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [25] Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. Faithful chain-of-thought reasoning. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the*

Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), 2023.

- [26] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.
- [27] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [28] Liangming Pan, Xiaobao Wu, Xinyuan Lu, Luu Anh Tuan, William Yang Wang, Min-Yen Kan, and Preslav Nakov. Fact-checking complex claims with program-guided reasoning. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2023.
- [29] Jash Rajesh Parekh, Pengcheng Jiang, and Jiawei Han. Structure-augmented reasoning generation. *arXiv preprint arXiv:2506.08364*, 2025.
- [30] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [31] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [32] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [33] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, 2025.
- [34] Jimeng Shi, Sizhe Zhou, Bowen Jin, Wei Hu, Runchu Tian, Shaowen Wang, Giri Narasimhan, and Jiawei Han. Hypercube-based retrieval-augmented generation for scientific question-answering. *arXiv preprint arXiv:2505.19288*, 2025.
- [35] Jimeng Shi, Wei Hu, Runchu Tian, Bowen Jin, Wonbin Kweon, SeongKu Kang, Yunfan Kang, Dingqi Ye, Sizhe Zhou, Shaowen Wang, et al. Multicube-rag for multi-hop question answering. *arXiv preprint arXiv:2602.15898*, 2026.
- [36] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- [37] Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerossearch: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*, 2025.
- [38] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo, Yang Wang, Yaming Liang, Xiangyang Ling, Jie Zhou, Shaoliang Cai, and Jin Luo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=nnV01PvbTv>.
- [39] Jiashuo Sun, Pengcheng Jiang, Saizhuo Wang, Jiajun Fan, Heng Wang, Siru Ouyang, Ming Zhong, Yizhu Jiao, Chengsong Huang, Xueqiang Xu, Pengrui Han, Peiran Li, Jiaxin Huang, Ge Liu, Heng Ji, and Jiawei Han. Rethinking the reranker: Boundary-aware evidence selection for robust retrieval-augmented generation, 2026. URL <https://arxiv.org/abs/2602.03689>.

- [40] Jiashuo Sun, Shixuan Liu, Zhaochen Su, Xianrui Zhong, Pengcheng Jiang, Bowen Jin, Peiran Li, Weijia Shi, and Jiawei Han. GRACE: Generative representation learning via contrastive policy optimization. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=hs9lwjH1bJ>.
- [41] Jiashuo Sun, Yixuan Xie, Jimeng Shi, Shaowen Wang, and Jiawei Han. Tasr-rag: Taxonomy-guided structured reasoning for retrieval-augmented generation. *arXiv preprint arXiv:2603.09341*, 2026.
- [42] Jiashuo Sun, Xianrui Zhong, Sizhe Zhou, and Jiawei Han. DynamicRAG: Leveraging outputs of large language model as feedback for dynamic reranking in retrieval-augmented generation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=NuCTKof1sV>.
- [43] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.
- [44] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023.
- [45] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35, 2022.
- [46] Junlin Wu, Xianrui Zhong, Jiashuo Sun, Bolian Li, Bowen Jin, Jiawei Han, and Qingkai Zeng. Structure-r1: Dynamically leveraging structural knowledge in llm reasoning through reinforcement learning, 2025. URL <https://arxiv.org/abs/2510.15191>.
- [47] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [48] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.
- [49] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [50] Xuhui Zheng, Kang An, Ziliang Wang, Yuhang Wang, and Yichao Wu. Stepsearch: Igniting llms search ability via step-wise proximal policy optimization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

A Limitations

While **PyRAG** demonstrates consistent gains across multi-hop benchmarks, our analysis (3.4) and case studies (Appendix G) reveal several limitations that we believe are informative for future work.

Retrieval recall is the upstream bottleneck. Our failure analysis (Figure 4b) shows that retrieval missing—where the retriever fails to surface gold evidence, accounts for roughly half of all incorrect predictions, making upstream retrieval recall the single largest source of failures. Although our adaptive retrieval mechanism mitigates this for sub-steps where the answer agent explicitly signals insufficient evidence, it cannot recover cases where retrieval silently returns plausible-looking but incorrect documents. Improving retrieval recall, for instance through query reformulation, learned retrievers, or hybrid sparse–dense retrieval, would yield the largest single accuracy gain and is largely orthogonal to **PyRAG**’s contributions at the planning and execution layers.

Answer agents struggle to utilize retrieved context. Although our RL fine-tuning of the Answer Agent yields measurable improvements over the training-free variant, a substantial fraction of remaining failures still trace back to this stage, both as intermediate error propagation in Figure 4b and as the dominant “Unknown Error” mode in Figure 4c, where the program executes successfully but the answer agent cannot ground a response in the retrieved passages. The executable interface successfully isolates failures to a specific stage, but the underlying difficulty, faithfully grounding answers in retrieved evidence and composing them across hops, is not eliminated by current RL objectives. Improving how language models exploit retrieved context, for instance through evidence-grounded training signals, calibrated uncertainty expression, or aggregation-aware objectives, is a key direction for future work.

Brittleness of sentinel-based adaptive retrieval. The execution-driven adaptive retrieval mechanism is triggered by string-level matching against sentinel responses such as “unknown” or “cannot answer.” Failure F2 illustrates a concrete weakness: when a sentinel value is interpolated into a downstream query as if it were content, retrieval errors silently propagate. A more robust design would replace string sentinels with structured return types or calibrated confidence signals.

Under-decomposition by the planner. Among program errors, the dominant mode is the silent single-retrieve case, where the planner emits syntactically valid code that issues only one `retrieve()` call for a question that requires multiple hops. This bypasses the reasoning chain entirely and is invisible to compiler-grounded self-repair, since no exception is raised. Plan complexity estimation, or an auxiliary objective penalizing under-decomposition, would be a natural complementary signal.

B Broader Impacts

PyRAG aims to improve the factual grounding and interpretability of multi-hop question answering by replacing opaque natural-language reasoning trajectories with executable programs whose intermediate states are inspectable. We see two main positive impacts: (1) the inspectable execution trace lowers the barrier to auditing model behavior in knowledge-intensive applications, where silent reasoning errors or fabricated intermediate facts are otherwise difficult to localize; and (2) the program-execution interface decouples retrieval, computation, and aggregation, allowing deterministic operations (e.g., date arithmetic, boolean conjunction) to be handled outside the language model and reducing a known source of hallucination. At the same time, PyRAG inherits the risks of any retrieval-augmented system. Because final answers are grounded in retrieved passages, biases, factual errors, or under-representation of certain groups, languages, or domains in the underlying corpus can propagate into outputs while appearing well-supported by an inspectable trace—potentially lending unwarranted credibility to incorrect conclusions. The structured planning interface could also, in principle, be repurposed to automate the generation of seemingly evidence-backed but misleading content at scale. Finally, executing model-generated code introduces a standard but non-trivial security surface: deployments must sandbox the Python interpreter and restrict tool APIs to prevent untrusted programs from performing unintended actions. We restrict the runtime in our experiments to the two tool primitives `retrieve` and `answer` over a fixed Wikipedia corpus, and we encourage similar isolation in any downstream deployment.

C Extended Related Work

Multi-Hop Retrieval-Augmented Generation. Multi-hop QA requires chaining evidence across multiple passages, which vanilla RAG [23] cannot handle in a single retrieval step. Iterative prompting-based methods interleave retrieval with chain-of-thought (CoT) reasoning [45] and reasoning-action loops [49] or decomposed sub-questions [30, 44, 16, 31, 19, 34, 35]. A parallel line of graph-based approaches constructs reasoning structures over retrieved content [5, 10, 1, 29, 38, 42, 40, 46, 41, 39]. More recent work involves training the search policy with reinforcement learning, optimizing the multi-turn retrieval process end-to-end. Search-R1 [17] extends DeepSeek-R1 [9] style training to retrieval with outcome-based rewards and retrieved token masking to stabilize multi-turn updates, and R1-Searcher [36] similarly incentivizes search invocation through outcome-based RL. StepSearch [50] densifies the RL signal via hop-wise rewards and redundancy penalties. Training-free agentic variants, such as Search-o1 [24], embed retrieval inside o1-style long CoT, distilling retrieved documents before reinjecting them into the reasoning chain. Across these methods, the retrieval-reasoning interaction remains an implicit trajectory shaped by prompts or rewards, and error detection relies on LLM-generated signals, rather than external verification. **PyRAG** instead represents the full pipeline as an executable program, making the reasoning structure explicit, dynamic, and verifiable via compiler feedback.

C.1 Program-Guided Reasoning

Executable code has proven to be effective for reasoning tasks with well-defined symbolic structure. PAL [7] and Program-of-Thoughts [3] offload numerical reasoning to a Python interpreter, separating planning from deterministic execution. Binder [4] extends this to table QA via unified natural-language and SQL commands, Faithful-CoT [25] translates questions into symbolic programs that are then executed by an external solver, and Logic-LM [27] couples an LLM front-end with symbolic solvers for logical reasoning. ProgramFC [28] compiles natural-language claims into Python-style verification, which are then executed by fact-checking modules. These approaches assume that the evidence required for reasoning is available a priori, grounded in self-contained inputs such as tables or closed evidence corpora. A complementary line of work, exemplified by DSPy [20], treats LM pipelines as compilable programs and automatically optimizes their prompts and demonstrations via bootstrapped traces; its HotPotQA case study uses a hand-designed two-hop module with fixed structure. DSPy operates at the level of *pipeline construction and prompt optimization*, whereas **PyRAG** prescribes a specific *reasoning representation*—a dynamically generated executable program per query—with execution-grounded self-repair and adaptive retrieval as runtime mechanisms; the two are in principle composable. We do not include DSPy as a direct empirical baseline because the two systems target different layers of the stack and rely on incompatible optimization regimes. DSPy’s published HotpotQA pipeline is a hand-designed two-hop module with fixed structure, which cannot adapt to questions of varying hop counts (e.g., MuSiQue’s mixture of 2–4 hop queries) without manual redesign. More fundamentally, DSPy’s strength comes from teleprompter-based prompt and demonstration bootstrapping over a labeled training set—disabling this optimization reduces DSPy to a standard ReAct prompt, while enabling it makes the comparison incommensurable with our training-free setting and operates at a different granularity (prompt-level optimization) than our RL-trained variant (policy-level optimization). We instead view PyRAG and DSPy as composable: PyRAG prescribes the per-query reasoning representation, while DSPy could in principle optimize the prompts of PyRAG’s individual agents. We leave this integration to future work. **PyRAG** targets a fundamentally different setting, open-domain multi-hop QA, where intermediate answers are unknown at synthesis time and must be dynamically retrieved during execution, with later retrieval queries depending on the results of earlier ones.

D Algorithm

E Additional Experiment

E.1 Implement Details

PyRAG is implemented as a three-agent pipeline: a **Decompose Agent** that breaks the input question into atomic sub-queries (JSON list, with up to 3 self-correction retries); a **Plan Agent** that trans-

Algorithm 1 PyRAG

Require: Question q ; retriever $\mathcal{R}$; tool APIs $\text{retrieve}(\cdot, k)$, $\text{answer}(\cdot, \cdot)$; agents $\mathcal{A}_{\text{dec}}$, $\mathcal{A}_{\text{plan}}$, $\mathcal{A}_{\text{ans}}$; default top- k k_0 , boosted top- k k_1 ($k_1 > k_0$); max repair rounds T ; sentinel set $\mathcal{S} = \{\text{"unknown"}, \text{"cannot answer"}, \dots\}$

Ensure: Final answer $\hat{a}$; execution trace τ

```
1: // Stage 1: Decomposition
2:  $\mathbf{s} = [s_1, \dots, s_n] \leftarrow \mathcal{A}_{\text{dec}}(q)$  ▷ atomic sub-queries
3: // Stage 2: Program Synthesis
4:  $\pi \leftarrow \mathcal{A}_{\text{plan}}(q, \mathbf{s})$  ▷ executable Python program over {retrieve, answer}
5: // Stage 3: Execution with grounded refinement
6:  $t \leftarrow 0$ ,  $\tau \leftarrow \emptyset$ ,  $\text{env} \leftarrow \text{PythonInterpreter}()$ 
7: while  $t \leq T$  do
8:    $\hat{a}, \tau, \text{err} \leftarrow \text{EXECUTE}(\pi, \text{env}, \mathcal{R}, \mathcal{A}_{\text{ans}}, k_0, k_1, \mathcal{S})$ 
9:   if  $\text{err} = \text{None}$  then
10:    break ▷ program executed successfully
11:   else
12:     $\pi \leftarrow \mathcal{A}_{\text{plan}}(q, \mathbf{s}, \pi, \text{err})$  ▷ (A) compiler-grounded self-repair
13:     $t \leftarrow t + 1$ 
14:   end if
15: end while
16: return  $\hat{a}, \tau$ 

17: Procedure EXECUTE( $\pi, \text{env}, \mathcal{R}, \mathcal{A}_{\text{ans}}, k_0, k_1, \mathcal{S}$ ):
18:   try:
19:     for each statement  $\ell \in \pi$  do
20:       if  $\ell$  is  $v \leftarrow \text{retrieve}(\text{query}, k)$  then
21:          $\text{env}[v] \leftarrow \mathcal{R}(\text{query}, k)$ 
22:       elseif  $\ell$  is  $v \leftarrow \text{answer}(\text{query}, \text{docs})$  then
23:          $a \leftarrow \mathcal{A}_{\text{ans}}(\text{query}, \text{docs})$ 
24:         if  $a \in \mathcal{S}$  then ▷ (B) execution-driven adaptive retrieval
25:            $\text{docs}' \leftarrow \mathcal{R}(\text{query}, k_1)$  ▷ boost top- $k$  for under-evidenced step
26:            $a \leftarrow \mathcal{A}_{\text{ans}}(\text{query}, \text{docs}')$ 
27:            $\text{env}[v] \leftarrow a$ 
28:         else ▷ native Python ops (regex, arithmetic, control flow)
29:           evaluate  $\ell$  in  $\text{env}$ 
30:           append  $(\ell, \text{env}[v])$  to  $\tau$ 
31:       return  $\text{env}[\text{final}], \tau, \text{None}$ 
32:   except Exception  $e$ : return  $\text{None}, \tau, e$ 
```

lates the sub-queries into executable Python code using two primitive functions, `retrieve()` and `answer()`; and an **Answer Agent** that processes each `answer()` call by conditioning on retrieved passages enclosed in structured `<answer>` tags. All inter-agent communication is mediated through a shared `execution_log` that records every retrieval and QA step.

Runtime errors in LLM-generated code trigger a self-repair loop in which the Plan Agent is re-prompted with the failed code and the Python traceback, up to `MAX_FIX_ROUNDS=3` attempts. Syntax errors detected during code generation are corrected inline within the same generation call, also up to 3 retries.

All language models are served with vLLM [22]. The Plan Agent uses Qwen2.5-Coder-7B-Instruct [15] (tensor parallel size 2); the Decompose and Answer Agents use Qwen2.5-7B-Instruct [47] (tensor parallel size 2). For 72B-backbone experiments, Qwen2.5-72B-Instruct is substituted with tensor parallel size 4.

We fine-tune all three agents with GRPO [32] using the VERL framework [33] under a shared-parameter, curriculum-style schedule: a single backbone is sequentially specialized into the Answer, Plan, and Decompose roles, with the other two agents frozen at each stage. The order is deliberate: the Answer Agent is trained first since it is the terminal step of every reasoning chain and its quality bounds the end-to-end reward; the Plan Agent is trained next on top of a well-calibrated answerer,

Table 4: Dataset statistics for training and evaluation.

Split	Dataset	#Examples	Task Type	Domain
Train	NQ	79,168	Single-hop	In-domain
	HotpotQA	8,757	Multi-hop	In-domain
	<i>Total</i>	87,925	–	–
Eval	HotpotQA	7,405	Multi-hop	In-domain
	PopQA	14,267	Single-hop	Out-of-domain
	2WikiMultiHopQA	12,576	Multi-hop	Out-of-domain
	Musique	2,417	Multi-hop	Out-of-domain
	Bamboogle	125	Multi-hop	Out-of-domain

so program-level credit assignment is conditioned on a reliable execution backend; finally, the Decompose Agent is trained against frozen Plan and Answer Agents that are both already strong, which substantially reduces the variance of the end-to-end reward signal. The reward is a weighted combination of EM and F1, $r = 0.7 \cdot \text{F1} + 0.3 \cdot \text{EM}$, computed by executing the full pipeline against gold answers. All three agents are fine-tuned with LoRA [13] (rank 64, $\alpha = 32$). The Answer Agent uses learning rate $1\text{e-}6$, cosine schedule, rollout $n = 8$, batch size 32, 1 epoch. The Plan and Decompose Agents use batch size 64, rollout $n = 4$, learning rate $3\text{e-}6$, KL penalty $\lambda = 0.001$ (low-variance KL), 2 epochs each. All RL experiments are conducted on a single node of $8 \times \text{A100}$ 80 GB GPUs.

E.2 Datasets

Following the data setup of Search-R1 [17], we train on a mixture of Natural Questions (NQ) [21] and HotpotQA [48], yielding 87,925 training examples in total (79,168 single-hop NQ and 8,757 multi-hop HotpotQA). This mixture exposes the model to both single-hop factoid retrieval and compositional multi-hop reasoning during RL fine-tuning, while keeping the training distribution comparable to prior RL-trained RAG baselines for fair comparison.

For evaluation, we use seven datasets grouped along two axes domain (in- vs. out-of-domain relative to training) and hop count (single- vs. multi-hop). The in-domain evaluation sets are the held-out splits of NQ (3,610) and HotpotQA (7,405). For out-of-domain evaluation, we include three single-hop benchmarks: PopQA [26] (14,267), and three multi-hop benchmarks, 2WikiMultiHopQA [12] (12,576), MuSiQue [43] (2,417), and Bamboogle [30] (125). The multi-hop out-of-domain sets in particular stress-test whether the structured planning prior learned by PyRAG transfers across question distributions, hop counts (MuSiQue contains 2–4 hop questions), and compositional patterns unseen during training. Detailed statistics are reported in Table 4. We use Exact Match (EM) as the primary metric throughout, consistent with prior work.

F Prompts

This appendix lists the full prompts used by the three PyRAG agents (Decompose, Plan, Answer) together with the two repair templates triggered by compiler-grounded self-repair. Each prompt is reproduced verbatim from our implementation; placeholders in braces (e.g. `{original_query}`) are filled in at runtime. We use three colour codes throughout: **blue** for system prompts that fix an agent’s role and output schema, **green** for user-side templates that supply per-question context, and **orange** for repair templates triggered by execution failures.

F.1 Decompose Agent

The Decompose Agent maps the original multi-hop question q to a list of atomic, single-hop sub-queries $s = [s_1, \dots, s_n]$. The system prompt fixes a strict JSON-list output schema so the result can be parsed deterministically and consumed by the Plan Agent without further post-processing; the user prompt supplies the original question together with a one-shot example that anchors the expected

granularity (one search-engine-answerable claim per item). Parsing failures trigger up to three retries before falling back to using the original question as a single-element list.

Decompose Agent	SYSTEM PROMPT
You are a question decomposition agent for multi-hop QA. Break a complex question into a minimal list of atomic single-hop sub-queries. Each sub-query should be independently answerable by a search engine. Return a JSON list of strings ONLY. No explanation, no markdown, no extra text.	
Decompose Agent	USER TEMPLATE
Original question: <code>{query}</code> Return a JSON list of atomic sub-queries. Example: <code>["Who directed Inception?", "Who directed Jurassic Park?", "When was Christopher Nolan born?"]</code>	

Figure 5: Prompts used by the Decompose Agent. The system prompt enforces a parseable JSON-list contract; the user template supplies the question and a one-shot example fixing sub-query granularity.

F.2 Plan Agent

The Plan Agent is the core of PyRAG: given the original question q and the decomposed sub-queries s , it synthesises an executable Python program over the two tool primitives `retrieve(query)` and `answer(query, docs)`. The system prompt (Figure 6) codifies the executable interface as a contract: it specifies the exact function signatures, including a no-docs aggregation mode for the final synthesis call; the data-flow discipline that intermediate results must be bound to identifiers and reused via f-strings rather than re-derived; and the two-part synthesis format “*Given: <facts>. Answer the question: <original question verbatim>*” that prevents an intermediate answer from leaking into the question template. The user prompt instantiates this contract for the specific query and supplies a worked one-shot example demonstrating variable threading across hops.

F.2.1 Self-Repair Templates

PyRAG’s compiler-grounded self-repair operates at two granularities. *Syntax-level* feedback (Figure 9) is triggered when the model’s output fails to compile under `compile(..., 'exec')`; the failed snippet and the parser’s error location are returned to the same generation call, up to three retries. *Runtime-level* feedback (Figure 8) is triggered after a successful compile when the program raises a Python exception during execution; the original question, the failing program, and the traceback are surfaced back to the Plan Agent, which produces a corrected program for the runtime to re-execute. Both templates re-iterate the contract violations most commonly responsible for failure (uninitialised `final_answer`, parsed `answer()` return values, missing `docs` arguments) so that repair is grounded in deterministic compiler signals rather than the model’s self-judgement.

F.3 Answer Agent

The Answer Agent is invoked once per `answer(query, docs)` call in the executed program and operates in two distinct modes that share an identical `<redacted_thinking>/<answer>` output schema. In *evidence mode* (Figure 10), the agent receives a sub-query together with retrieved passages, must answer using only those passages, and must cite each used passage inline as “Doc [i]”. The schema fixes type-matching between the question and the answer span (a *who* question must return a name, not a date) and reserves the literal token “unknown” as a sentinel for under-evidenced steps, which directly drives the execution-driven adaptive retrieval mechanism described in Section 2.6 of the main text. In *aggregation mode* (Figure 11), no documents are

supplied; the prompt instead relies on the two-part *Given: ... Answer the question: ...* template emitted by the Plan Agent, and the agent composes the final answer from the supplied facts. The aggregation prompt explicitly forbids yes/no responses to *wh*-questions, addressing the failure mode in which the model otherwise treats the synthesis call as fact verification. Retrieved documents are formatted as [Doc 1] \n<text>\n\n[Doc 2] \n<text>\n\n... so that inline citation indices are unambiguous.

G Case Study

You are a planning agent that writes Python code to answer questions via RAG.

You have exactly two functions available:

```
retrieve(query: str) -> List[str]
    Searches a retrieval system and returns a list of relevant
    document strings. Do NOT pass a topk argument; the retrieval
    count is controlled externally.

answer(query: str, docs: List[str]) -> str
    Calls an LLM to answer a question given a list of documents.
    Returns ONLY the short text inside the model's <answer>...
    </answer> block (the runtime extracts it). Use that return
    value directly in f-strings -- it is a short phrase, not the
    full model reply.

answer(query: str) -> str (no docs -- aggregation / synthesis mode)
    Same as above but with NO documents. The model answers using
    ONLY the information already present in the query string. Use
    this for the FINAL aggregation step.
```

Rules:

- Use retrieve() + answer() step by step to answer each sub-query.
- If you call retrieve() for a sub-question, pass that return value into the matching answer() for that sub-question.
- Build on intermediate answers by interpolating them into later queries using f-strings.
- Variable names MUST be valid Python identifiers: only letters, digits, and underscores, NO spaces.
- Every variable you reference MUST be assigned BEFORE it is used.
- Do NOT parse answer() return values in generated code (no .split(), regex, or indexing). The runtime already returns the short <answer> span.
- If you use a for/if/while block that might conditionally set final_answer, initialise final_answer = "" BEFORE the block.
- The last line of your code MUST be:


```
final_answer = answer(<synthesis question>)
```

 Do NOT pass docs to this final answer() call -- aggregation only.
- NEVER use an f-string or string concatenation as final_answer. final_answer MUST ALWAYS be the return value of an answer() call.
- CRITICAL: The <synthesis question> must use this TWO-PART format:


```
"Given: <fact1>, <fact2>, ...
  Answer the question: <ORIGINAL QUESTION>"
```

 where the first part lists intermediate results as background facts and the second part REPEATS the ORIGINAL question VERBATIM after "Answer the question:". Do NOT rephrase the original question or embed intermediate answers INTO the question itself.


```
BAD: f"Which American director, {name}, who is {nationality},
      hosted ...?" (answer leaks into the question)
      GOOD: f"Given: {name} hosted the event, {name} is {nationality}.
      Answer the question: Which American director hosted ...?"
```
- Valid Python 3 only: use ASCII double or single quotes for strings; never put backticks around variable names or expressions.
- Return ONLY the Python code, no explanation, no imports, no markdown.

Figure 6: Plan Agent system prompt. Codifies the executable interface as a contract — function signatures, data-flow discipline, and the two-part synthesis format that prevents answer leakage into the final question.

Plan Agent	USER TEMPLATE
	Original question: <code>{original_query}</code> Sub-queries to resolve: <code>{sub_queries}</code> Reference example (do NOT copy, write code for the actual question above): <code>{CODE_EXAMPLE}</code> Now write the Python code for the original question. End with: <code>final_answer = answer(f"Given: <facts>. Answer the question: {original_query}")</code>
Plan Agent	ONE-SHOT EXAMPLE (CODE_EXAMPLE)
	<pre> # Example for: "Who was born earlier, the director of # Inception or Jurassic Park?" docs1 = retrieve("Who directed Inception?") director1 = answer("Who directed Inception?", docs1) docs2 = retrieve("Who directed Jurassic Park?") director2 = answer("Who directed Jurassic Park?", docs2) docs3 = retrieve(f"When was {director1} born?") birth1 = answer(f"When was {director1} born?", docs3) docs4 = retrieve(f"When was {director2} born?") birth2 = answer(f"When was {director2} born?", docs4) # ALWAYS end with answer() -- list facts first, # then repeat the ORIGINAL question verbatim final_answer = answer(f"Given: {director1} directed Inception and was born {birth1}, " f"{director2} directed Jurassic Park and was born {birth2}. " f"Answer the question: Who was born earlier, the director of " f"Inception or Jurassic Park?") </pre>

Figure 7: Plan Agent user-side context. Top: the per-question template filled with the original query, the decomposed sub-queries, and the one-shot CODE_EXAMPLE. Bottom: the example itself, demonstrating variable threading across hops and the two-part synthesis call.

Plan Agent	RUNTIME-ERROR REPAIR TEMPLATE
The following Python code was generated to answer the question but raised a runtime error. Fix the code so it runs correctly. Return ONLY the corrected Python code. <pre> === Original question === {original_query} === Failing code === {failed_code} === Runtime error === {error_msg} === Fix instructions === - Do not parse answer() return values in generated code; the runtime returns short <answer> text. - Every variable you reference must be explicitly assigned earlier. - If retrieve() was called, pass its docs into answer(); do not use answer(..., []) for that step. - Initialise final_answer = "" before any loop/conditional that might set it. - The last line MUST be: final_answer = answer(<synthesis question>) -- call answer() with NO docs to aggregate all intermediate results. NEVER use an f-string or string as final_answer directly. </pre>	

Figure 8: Runtime-level self-repair template. Triggered when an executed program raises a Python exception; the traceback and the original question are surfaced back as deterministic, grounded feedback signals.

Plan Agent	SYNTAX-ERROR REPAIR TEMPLATE
<pre> --- Your previous output is NOT valid Python (syntax error). Details: {error_detail} Fix it. Use only ASCII ' or " for strings; never wrap variable names in backticks; output code only. Previous attempt: ```python {failed_code} ``` Return ONLY corrected Python code. </pre>	

Figure 9: Syntax-level self-repair template. Triggered when the generated code fails to compile; the parser's error location is fed back to the same generation call.

Answer Agent	EVIDENCE MODE (WITH DOCUMENTS)
You are given a question and retrieved documents. You MUST answer using ONLY information from the retrieved documents. Even for yes/no questions, decide yes or no by reasoning from facts in the documents. Output format (STRICT): <thinking> ... </thinking> <answer> ... </answer> Evidence citation rule: - Whenever you use evidence from the documents in your reasoning, you MUST cite it inline as Doc [i] (matching the document indices shown in the retrieved block, e.g. [Doc 1] -> Doc [1]). - Only cite documents that are actually relevant. - Keep <thinking> concise (1-3 sentences). Answer rules: - The <answer> should be a short phrase, preferably taken directly from the documents when possible. - Match the answer TYPE to the QUESTION: - WHO / which person / born first / earlier -> a person's NAME in <answer>, not only a date; - WHEN -> date or time; - yes/no -> exactly yes / no / unknown when the documents do not support a definite answer. - Do NOT output anything outside <thinking> and <answer>. Example (do NOT copy the content, only follow the style): <thinking>Doc [1] states that Future Ted serves as the narrator, and Doc [4] confirms the voice actor.</thinking> <answer> Ted Mosby </answer>	

Figure 10: Answer Agent system prompt — *evidence mode*. Used when at least one retrieved passage is supplied. The schema fixes question-type matching and reserves “unknown” as the sentinel that drives adaptive retrieval.

Answer Agent	AGGREGATION MODE (NO DOCUMENTS)
There are NO retrieved documents. The question text itself contains background facts (after 'Given:') and the actual question to answer (after 'Answer the question:'). You MUST use the provided facts to answer the ACTUAL QUESTION. CRITICAL: Your job is to ANSWER the question, NOT to confirm whether the facts are true. - If the question asks WHO / WHICH person -> reply with a person's NAME. - If the question asks WHEN -> reply with a date or time. - If the question asks WHERE -> reply with a place. - ONLY answer yes/no when the question is explicitly a yes/no question (e.g. 'Are both ...?', 'Is it true ...?'). - NEVER answer 'yes' or 'no' to a WHO/WHICH/WHEN/WHERE question. Output format (STRICT): <pre><thinking> ... </thinking> <answer> ... </answer></pre> - In <thinking>, identify the actual question type and combine the given facts to produce the answer (1-2 sentences). - The <answer> must directly answer the question -- a name, date, place, etc. -- NOT 'yes' or 'no' unless the question is truly yes/no. - Do NOT output anything outside <thinking> and <answer>.	

Figure 11: Answer Agent system prompt — *aggregation mode*. Used when the docs argument is empty, i.e. in the final synthesis call. Forbids yes/no responses to *wh*-questions, eliminating the failure mode where the synthesis call collapses into fact verification.

Case A	Correct
Question: 2014 S/S is the debut album of a South Korean boy group that was formed by who? Gold: YG Entertainment Predicted: YG Entertainment <pre># Step 1-2: identify the boy group ctx_grp = retrieve("2014 S/S debut South Korean boy group", topk=5) group = answer("Which group is the album by?", ctx_grp) # group = "WINNER" # Step 3-4: the next query *uses* the bound variable 'group' ctx_co = retrieve(f"Which company formed {group}?", topk=5) company = answer(f"Which company formed {group}?", ctx_co) # company = "YG Entertainment" # Step 5: final aggregation is itself an answer(...) call whose query # is built from the two intermediate variables final = answer(f"Given: {group} is a South Korean boy group; " f"{company} formed {group}. " f"Answer the question: 2014 S/S is the debut album " f"of a South Korean boy group that was formed by who?", ctx_co,) # final -> "YG Entertainment" [ok]</pre>	

Figure 12: A representative correct example. Variables produced at one step are explicitly consumed by subsequent calls through string interpolation.

Case B	Correct
Question: Which other Mexican Formula One race car driver has held the podium besides the Force India driver born in 1990? Gold: Pedro Rodríguez Predicted: Pedro Rodríguez	
<pre> # Step 1-2: a known Mexican F1 podium driver ctx_a = retrieve("Mexican F1 driver who held the podium", topk=5) known = answer("Who is the Mexican F1 podium driver?", ctx_a) # known = "Sergio P\`erez" # Step 3-4: try to identify the Force India 1990-born driver ctx_b = retrieve("Force India driver born in 1990", topk=5) exclude = answer("Which Force India driver was born in 1990?", ctx_b) # exclude = "unknown" [warn] insufficient evidence </pre>	
<pre> # Step 3-4: executor reissues a broader retrieval (adaptive re-retrieval) ctx_b = retrieve("Force India driver born in 1990", topk=10) exclude = answer("Which Force India driver was born in 1990?", ctx_b) # exclude = "Pedro Rodr\`iguez and Sergio P\`erez" </pre>	
<pre> # Step 5: the final aggregation is again an answer(...) call, # whose query templates in the previously bound variables final = answer(f"Given: {known} is a Mexican F1 podium driver; " f"the Mexican F1 podium drivers are {exclude}. " f"Which other Mexican F1 podium driver is there besides {known}?", ctx_c,) # final -> "Pedro Rodr\`iguez" [ok] </pre>	

Figure 13: When Step 4 returns the sentinel "unknown", execution-guided refinement triggers a broader re-retrieval (Step 5–6, highlighted). The plan structure is preserved; only the under-evidenced sub-step is repaired. Adaptive retrieval recovers from an under-evidenced sub-step without modifying the overall plan, illustrating the benefit of execution-grounded refinement.

Case C	Correct
Question: <i>Are Freakonomics and In the Realm of the Hackers both American documentaries?</i> Gold: no	Predicted: No
<pre> films = ["Freakonomics", "In the Realm of the Hackers"] predicates = ["is a documentary", "is American"] # Two predicates per film -> grid of 4 yes/no probes flags = {f: {} for f in films} for film in films: for pred in predicates: ctx = retrieve(f"Is {film} {pred}?", topk=5) flags[film][pred] = answer(f"Is {film} {pred}?", ctx) # flags = { # "Freakonomics": {"is a documentary": "no", "is American": "yes"}, # "In the Realm of the Hackers": {"is a documentary": "yes", "is American": "no"}, # } # Boolean structure handled in Python -- not by the answer agent def qualifies(d): return all(v.lower() == "yes" for v in d.values()) if all(qualifies(flags[f]) for f in films): final = "yes" else: final = "no" # final -> "no" (Freakonomics fails the documentary check; # In the Realm of the Hackers fails the American check) [ok] </pre>	

Figure 14: Boolean conjunction over a 2×2 grid of predicates. The plan reduces a “both X and Y” question to a Cartesian grid of yes/no probes whose conjunction is decided by the Python keyword `all`. The boolean structure is enforced by the program; the answer agent never has to perform multi-clause logical reasoning over a free-form prompt. The decision rule is expressed as a Python expression rather than delegated to the answer agent.

Case D	Correct
Question: <i>How old was Virginia Bruce when she starred in Let Freedom Ring?</i> Gold: 29	Predicted: 29
<pre> import re # Two retrievals supply the numeric inputs ctx_film = retrieve("In what year did Virginia Bruce star in Let Freedom Ring?", topk=5) year_str = answer("In what year did Virginia Bruce star in Let Freedom Ring?", ctx_film) # year_str = "1939" ctx_born = retrieve("In what year was Virginia Bruce born?", topk=5) born_str = answer("In what year was Virginia Bruce born?", ctx_born) # born_str = "1910" # Cast text -> int and compute the answer in Python; no further LLM call needed year_film = int(re.search(r"\d{4}", year_str).group()) year_born = int(re.search(r"\d{4}", born_str).group()) final = year_film - year_born # final -> 29 [ok] </pre>	

Figure 15: Arithmetic over retrieved values. The final answer is not contained in any retrieved document and cannot be produced by retrieve+answer alone—it is the *difference* of two retrieved years. PyRAG handles this by lifting the retrieved strings into Python integers via `int(...)` and computing the subtraction deterministically. An LLM doing arithmetic on natural-language dates inside a free-form prompt is a known source of error; here the computation is moved outside the model entirely. PyRAG separates retrieval (handled by tools) from computation (handled by Python), giving deterministic numeric answers without relying on LLM mental arithmetic.

Failure F1	Incorrect
Question: <i>Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?</i> Gold: keyboard function keys Predicted: iPod Touch, iPad ✗	
<pre># Step 1-2: correctly identify the program ctx_p = retrieve("Program the Apple Remote was designed for", topk=5) program = answer("What program is Apple Remote designed for?", ctx_p) # program = "Front Row" [ok]</pre>	
<pre># Step 3-4: planner SHOULD use 'program' here, but silently rebinds to "iTunes" ctx_d = retrieve("What other devices can control iTunes?", topk=5) # [warn] entity drift device = answer("What other devices can control iTunes?", ctx_d) # device = "iPod Touch, iPad"</pre>	
<pre># Step 5: final answer(...) propagates the corrupted variable final = answer(f"Given: Apple Remote was designed for {program}; " f"other devices that can control {program} include {device}. " f"Aside from the Apple Remote, what other device can control {program}?", ctx_d,) # final -> "iPod Touch, iPad" [wrong]</pre>	

Figure 16: Decomposition-stage entity drift. The Step 3 query should have been `f"...control {program}?"`, but the planning agent emitted the literal string "iTunes" instead. Because the plan exposes variables as first-class objects, the drift is precisely localizable to the planning stage—a free-form CoT trace would mix this error into surrounding reasoning text. The executable trace makes the failure point unambiguous.

Failure F2	HOTPOTQA id=47	Incorrect
Question: <i>According to the 2001 census, what was the population of the city in which Kirton End is located?</i> Gold: 35,124 Predicted: 12,790 ✗		
<pre> # Step 1-2: locality lookup succeeds ctx_loc = retrieve("Where is Kirton End located?", topk=5) locality = answer("Where is Kirton End located?", ctx_loc) # locality = "Kirton, Lincolnshire" # Step 3-4: the next hop (city containing the locality) FAILS ctx_city = retrieve(f"In what city is {locality}?", topk=10) city = answer(f"In what city is {locality}?", ctx_city) # city = "unknown" [warn] </pre>		
<pre> # Step 5-6: sentinel value "unknown" is interpolated as a *content* string ctx_pop = retrieve(f"Population of {city} in the 2001 census", topk=5) # [warn] pop = answer(f"Population of {city} in the 2001 census?", ctx_pop) # pop = "12,790" (population of an unrelated locality) </pre>		
<pre> # Step 7: final answer(...) silently consumes the corrupted hop final = answer(f"Given: Kirton End is located in {locality}; " f"the containing city is {city}; " f"the 2001 census population of {city} was {pop}. " f"What was the 2001 census population of the city in which Kirton End is located?", ctx_pop,) # final -> "12,790" [wrong] </pre>		

Figure 17: Retrieval failure that propagates because a sentinel value is treated as a content string. The string "unknown" returned at Step 4 is a sentinel meaning “no evidence,” but the plan treats it as a normal value and interpolates it into Step 5’s query. This points to a concrete fix: hops whose answer matches the "unknown" sentinel should branch into guarded fallback rather than continuing the data-flow chain. The executable trace localizes this to a single edge in the data dependency graph.

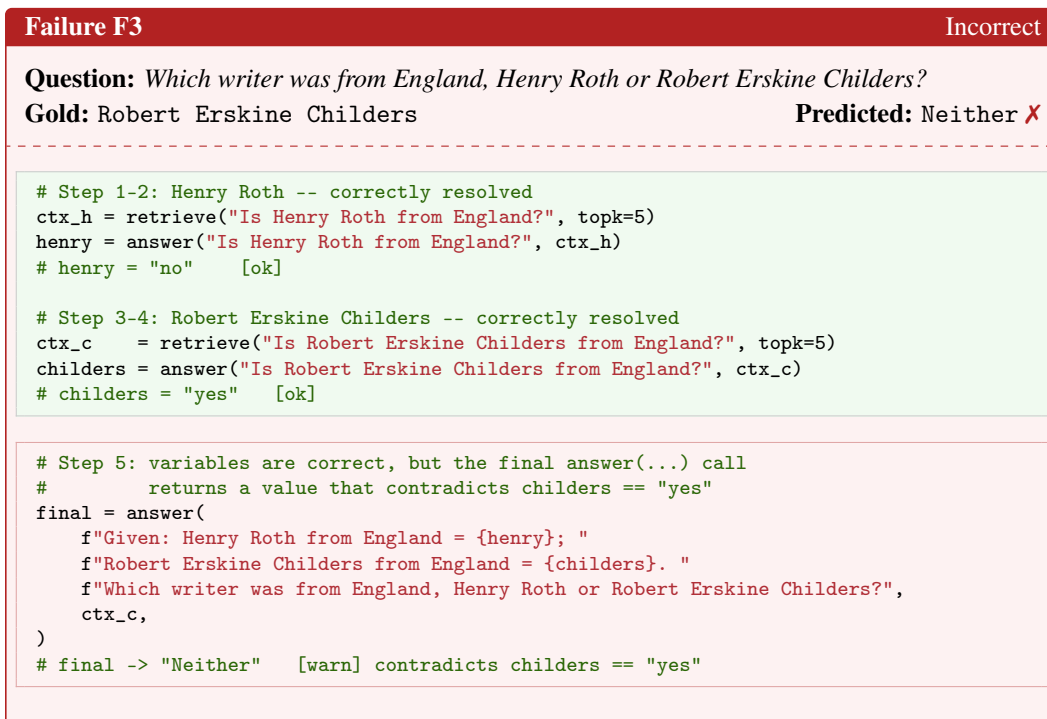


Figure 18: Final aggregation misreads its own variable bindings. Written as a Python program, the failure is sharply localized: every retrieved variable holds the right value, yet the final `answer(...)` call returns "Neither". The bug is therefore neither retrieval nor variable binding, it is the answer agent misreading its own variable bindings inside the final aggregation step. This isolates a clear bottleneck and motivates more structured aggregation prompts (e.g. typed slots) as future work. Because every variable is recorded in the trace, the contradiction between `childers = "yes"` and `final = "Neither"` is directly verifiable.

Failure F4	Incorrect
Question: <i>Are Northwestern University and Middlebury College both private schools?</i> Gold: yes Predicted: No ✗	
<pre> schools = ["Northwestern University", "Middlebury College"] flags = {} # Per-school predicate check -- both correctly resolved for s in schools: ctx = retrieve(f"Is {s} a private school?", topk=5) flags[s] = answer(f"Is {s} a private school?", ctx) # flags = {"Northwestern University": "yes", "Middlebury College": "yes"} [ok] </pre>	
<pre> # Final aggregation delegates the conjunction to the answer agent. # The model's thinking flips both variables to "public" and returns "No". final = answer(f"Given: Northwestern University is {flags[schools[0]]}, " f"Middlebury College is {flags[schools[1]]}. " f"Are Northwestern University and Middlebury College both private schools?", ctx,) # final -> "No" [warn] contradicts flags == {...: "yes", ...: "yes"} </pre>	
<pre> final = "yes" if all(v.lower() == "yes" for v in flags.values()) else "no" </pre>	

Figure 19: Boolean conjunction misexecuted by the answer agent. Both `flags` entries hold "yes", but the answer agent's reasoning trace narrates "*both Northwestern University and Middlebury College are public institutions*" before returning "No". The conjunction is the failure point—precisely the operation Python expresses with one keyword. Replacing the `final answer(...)` call with the one-line `all(...)` expression shown above eliminates the failure mode entirely. This is the inverse of Case C: when the boolean structure is enforced by the program rather than narrated to the LLM, the result is deterministic.

Failure F5	Incorrect
Question: Which of Aaron Goodwin's clients was born on May 31, 1984? Gold: Nate Robinson Predicted: None ✗	
<pre> ctx_cli = retrieve("Who are Aaron Goodwin's clients?", topk=5) clients = answer("List Aaron Goodwin's clienthttps://gasolsun36.github.io/PyRAG/ts.", ctx_cli) # clients is a *string*: "LeBron James, Dwight Howard, Chris Webber, ..." </pre>	
<pre> # The plan iterates 'clients' directly. In Python, iterating a 'str' # yields one *character* per step -- not one client. births = {} for name in clients: # [warn] name is "L", then "e", "B", "r", ... ctx = retrieve(f"When is the birthdate of {name}?", topk=5) births[name] = answer(f"When is the birthdate of {name}?", ctx) # Trace explodes: 400+ retrieve+answer calls, each on a single character. # Most return "unknown"; a few accidentally hit "August 6, 1992", "October 13, 1992", ... </pre>	
<pre> # Final aggregation cannot recover; no character matches "May 31, 1984" final = answer(f"Given: {births}. Which client was born on May 31, 1984?", ctx,) # final -> "None" [wrong] (correct answer: Nate Robinson) </pre>	
<pre> clients = [c.strip() for c in clients.split(",")] </pre>	

Figure 20: Type confusion in a for-loop. The bug is a single-line type confusion: `clients` is a comma-joined string, not a list, so iterating it character-by-character is silently legal Python and the executor fans out into hundreds of nonsensical retrievals on "L", "e", "B", ... The fix is the one-line cast shown above, after which the original for-loop iterates over actual client names. Such failures are uniquely visible in the executable trace: the explosion of single-character queries makes the type error mechanically obvious, whereas in a free-form CoT the same confusion would surface as “the model got distracted” or “hallucinated names.” The failure mode is unique to the executable interface, but so is the diagnosis—the trace pinpoints the exact line that needs a `.split(",")`.